Name Solution | No. . . . . | Mark. . . . .

CSC 211—Fundamental Algorithms and Data Structures

Growth functions and binary tricks

Term 1 Tutorial 3 — 25/26 February 2026

Programs can be classified in terms of the time they take and the space they occupy, depending on the size their input n. For, example if one sorts n numbers, the timing of the sorting is given in terms of this n and not in terms of the actual instructions that are executed to do the sorting or even in the time taken in years, months, days, hours, minutes, seconds or nanoseconds. Timing may be expressed as $T(n)$, the total time taken, e.g., $T(n) = 19 \cdot 2^n + 75n^2 + 119n + 23$. When the size of the input n gets very big, the value of $T(n)$ becomes so dominated by the 2^n term, that nothing else really matters. This asymptotic growth reflects the runtime behaviour of algorithms in terms of the time they take and the memory or storage space they occupy. This tends to be given as an upper bound of the time taken, $O(g(n))$, or a lower bound, $\Omega(g(n))$, or a so-called tight bound $\Theta(g(n))$. The usage of the bound $O(g(n))$ was introduced by Paul Bachmann in 1894 and was later expanded by Edmund Landau in 1909. When stating the order of a function $T(n) = f(n)$ as $g(n$, the most fashionable way is to say $f(n) \in O(g(n))$, despite many authors using $f(n)$ is $O(g(n))$ or $f(n) = O(g(n))$. Robert Sedgewick more correctly uses $T(n) \sim O(g(n))$.

Fill in the gaps.

1. Sometimes it is easy to express the time $T(n)$ an algorithm takes in terms of $T(n)$ itself, e.g., $T(n) = 2T(n) + 1$. Simplify this recursive expression so that only $T(n)$ appears on the left. What is its $O(g(n))$ order?

   See the notes/slides. [3]

2. Now try this one. Show that $T(n) = 2T(n/2) + cn$ is $O(n \log(n))$.
   [Hint: Let n be a power of n, and repeatedly halve it.]

   See the notes/slides. [3]

3. (a) Try this. First write this series $\sum_{r=1}^{\infty} 2^{-r}$ to several bits as a binary number, and then *without a calculator*, give its value.

   0,111...≈1 [3]

   (b) Prove that your answer is correct using the formula for $S_\infty = \sum_{r=1}^{\infty} p^{-r}$. Show how to derive the formula.

$$\begin{aligned}
\text{Let } S_n &= p^{-1} + p^{-2} \ldots + p^{-n}, \text{ then} \\
p^{-1}S_n &= \qquad p^{-2} + p^{-3} \ldots + p^{-n-1}, \text{ subtracting,} \\
(1-p^{-1})S_n &= p^{-1} - p^{-n-1}, \\
S_n &= \frac{p^{-1} - p^{-n-1}}{1-p^{-1}},
\end{aligned}$$

and then since $p^{-n-1} \to 0 asn \to \infty$,

$$S_\infty = \frac{p^{-1}}{1-p^{-1}}.$$

Putting $p = 2$ gives $S_\infty = \frac{\frac{1}{2}}{1-\frac{1}{2}} = 1$

[3]

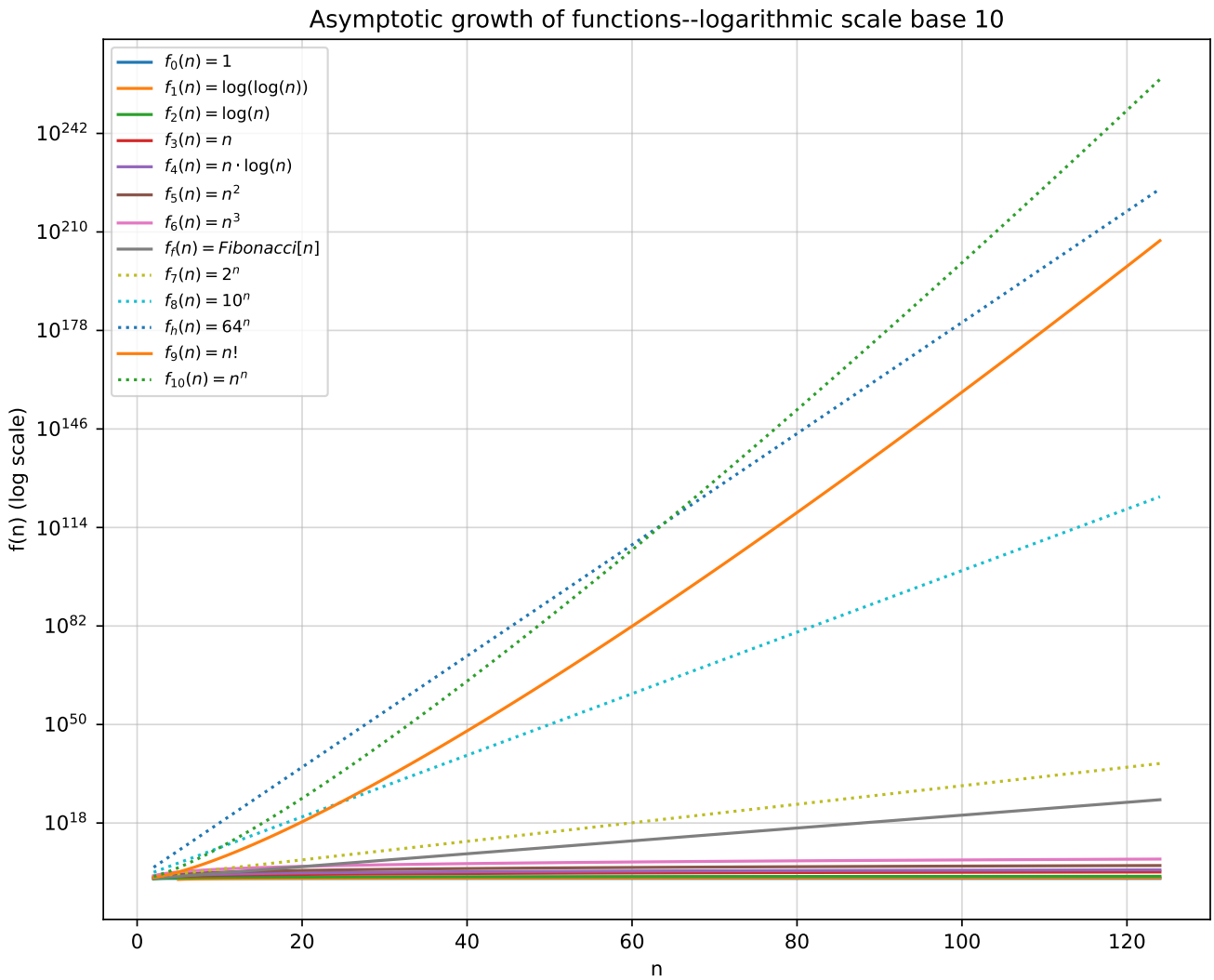
(c) Try this. Without a calculator, what is the value of $\sum_{r=0}^{\infty} 2^{-r}$?

$\boxed{1.111111111\ldots_2 = 2_{10}}$ [3]

(d) Try this without a calculator: what is the value of $\sum_{r=0}^{\infty} 7 \times 8^{-r}$?

$\boxed{7.777\ldots_8 = 111.111111111\ldots_2 = 8_{10}}$ [3]

ALGORITHM		1	2	3	4
Lines of C Code		8	7	14	7
Run time in microseconds		$3.4N^3$	$13N^2$	$46N \log_2 N$	$33N$
Time to solve a problem of size	10^2	3.4 secs	.13 secs	.03 secs	.003 secs
	10^3	.94 hrs	13 secs	.45 secs	.033 secs
	10^4	39 days	22 mins	6.1 secs	.33 secs
	10^5	108 yrs	1.5 days	1.3 mins	3.3 secs
	10^6	108 mill	5 mos	15 mins	33 secs
Max size problem solved in one	sec	67	280	2000	30,000
	min	260	2200	82,000	2,000,000
	hr	1000	17,000	3,500,000	120,000,000
	day	3000	81,000	73,000,000	2,800,000,000
If N multiplies by 10, time multiplies by		1000	100	10+	10
If time multiplies by 10, N multiplies by		2.15	3.16	10−	10


Asymptotic growth of functions--logarithmic scale base 10
$f_0(n) = 1$
$f_1(n) = \log(\log(n))$
$f_2(n) = \log(n)$
$f_3(n) = n$
$f_4(n) = n \cdot \log(n)$
$f_5(n) = n^2$
$f_6(n) = n^3$
$f_f(n) = Fibonacci[n]$
$f_7(n) = 2^n$
$f_8(n) = 10^n$
$f_h(n) = 64^n$
$f_9(n) = n!$
$f_{10}(n) = n^n$
10^{242}
10^{210}
10^{178}
10^{146}
10^{114}
10^{82}
10^{50}
10^{18}
f(n) (log scale)
0
20
40
60
80
100
120
n